```
import numpy as np
import pandas as pd
from IPython.display import display, Math

from mtx_utils import load_mtx

eps = 1e-3
A, b = load_mtx("tasks/1_5.txt")
n = len(b)

A, b
```

```
(array([[ 9.,  0.,  2.],
        [-6.,  4.,  4.],
        [-2., -7.,  5.]]),
 array([0., 0., 0.]))
```

QR разложение

```
def get_householder(v: np.ndarray):
    n = v.shape[0]
    denom = v @ v
    if np.isclose(denom, 0):
        return np.eye(n)
    return np.eye(n) - 2 * np.outer(v, v) / denom
```

$$QR = A.$$

```
def get_qr(A: np.ndarray):
    Ai = A.copy()
    Q = np.eye(n)

    for i in range(n - 1):
        b = Ai[:, i]
        v = np.zeros(n)
        for j in range(i, n):
            v[j] = b[j]
            if i == j:
                v[j] += np.sign(b[j]) * np.sqrt(np.sum(b * b))

        Hi = get_householder(v)

        Ai = Hi @ Ai
        Q @= Hi

    R = Ai.copy()
    return Q, R

Q, R = get_qr(A)
display(pd.DataFrame(Q), pd.DataFrame(R))
```

	0	1	2
0	-0.818182	-0.095042	0.567050
1	0.545455	-0.440184	0.713244
2	0.181818	0.892864	0.411991

	0	1	2
0	-1.100000e+01	0.909091	1.454545
1	6.379434e-16	-8.010780	2.513499
2	-9.654674e-16	-0.030962	6.047034

```
np.allclose(Q @ R, A, eps)
```

```
True
```

Поиск собственных значений

```
Ak = A.copy()
Qk, Rk = get_qr(Ak)
iterations = 0

found = np.zeros(n, dtype=bool)
block_found = np.zeros(n, dtype=bool)
lambdas = np.zeros(n, dtype=complex)

def check_real_eigen(Ak, j):
    return eps >= np.sqrt(np.sum(Ak[j + 1:, j] ** 2))

while not all(found):
    found = np.zeros(n, dtype=bool)
    j = 0
    iterations += 1

    while j < n:
        if j == n - 1:
            lambdas[j] = Ak[j, j]
            found[j] = True
            block_found[j] = False
            j += 1
            continue

        if check_real_eigen(Ak, j):
            lambdas[j] = Ak[j, j]
            found[j] = True
            block_found[j] = False
            j += 1
            continue

        block = Ak[j:j+2, j:j+2]
```

```
        b = block[0, 0] + block[1, 1]
        c = block[0, 0] * block[1, 1] - block[0, 1] * block[1, 0]

        d = b * b - 4 * c
        if d > -eps:
            block_found[j] = False
            j += 1
            continue

        l1 = (b + np.emath.sqrt(d)) / 2
        l2 = (b - np.emath.sqrt(d)) / 2

        if (
            block_found[j]
            and np.abs(l2 - lambdas[j]) <= eps
            and np.abs(l1 - lambdas[j + 1]) <= eps
        ):
            found[j] = found[j + 1] = True
        block_found[j] = True

        lambdas[j] = l2
        lambdas[j + 1] = l1

        j += 2

    Ak = Rk @ Qk
    Qk, Rk = get_qr(Ak)

display(pd.DataFrame(lambdas))
print(f"Iterations: {iterations}")
```

	0
0	10.027222+ 0.000000j
1	3.986389- 6.096106j
2	3.986389+ 6.096106j

```
Iterations: 29
```

```
def complex_to_latex(z, tol=1e-10):
    a = z.real
    b = z.imag

    if abs(a) < tol:
        a = 0.0
    if abs(b) < tol:
        b = 0.0

    if b == 0:
        return f"{a:.3f}"

    if a == 0:
        if b == 1:
            return "i"
```

```
        if b == -1:
            return "-i"
        return f"{b:.3f}i"

    sign = "+" if b > 0 else "-"
    b_abs = abs(b)

    if abs(b_abs - 1) < tol:
        return f"{a:.3f} {sign} i"
    return f"{a:.3f} {sign} {b_abs:.3f}i"

lambdas = np.sort_complex(lambdas)
for i, lam in enumerate(lambdas, start=1):
    display(Math(rf"\lambda_{{{i}}} = {complex_to_latex(lam)}"))
```

$\lambda_1 = 3.986 - 6.096i$

$\lambda_2 = 3.986 + 6.096i$

$\lambda_3 = 10.027$

Валидация

```
eigenvalues, eigenvectors = np.linalg.eig(A)

eigenvalues = np.sort_complex(eigenvalues)
print(eigenvalues)
```

```
[ 3.98636265-6.09624525j  3.98636265+6.09624525j 10.0272747 +0.j        ]
```

```
np.allclose(lambdas, eigenvalues, eps)
```

```
True
```